

JuliaCon and Remote Machines

Rohit Goswami · 2021-07-25

Using better machines for Jupyter-Julia workshops

Background

I haven't ever actually worked with `julia` before, however, some of the talks and workshops at this year's [Juliacon'21](#) were of great (applied) interest to me so I decided to follow along. In doing so, since my laptop is woefully inadequate for working through some of the materials (DFT), I decided to augment the installation instructions with remote machine usage notes.

Assumptions

The following gentle assumptions should not actually be much of a practical hurdle for this post, namely:

- You have a remote machine which runs some variant of Linux
 - No `nix` is assumed unfortunately
 - Should be capable of running `micromamba`
- Some code snippets assume a user-modifiable directory in the `$PATH`
 - Typically I assume something like `export PATH=$HOME/.local/bin:$PATH`

The code snippets arise in the [context of the DFT workshop](#)^[1], but are applicable across all the talks. Additionally, the snippets will assume **no forks** but in practice it is best to clone modifiable forks of these repositories so your mutations last forever.

Paths and Directories

For me, I use the following layout:

```
export jc21=$HOME/Git/Github/Julia/juliacon21
mkdir -p $jc21
cd $jc21
git clone git@github.com:mfherbst/juliacon_dft_workshop.git
```

Environment

We will start by setting up a `micromamba` environment. This is probably the most copied snippet on my site.

Now we can start with the (non-`julia`) scaffolding:

```
cd $jc21
micromamba create -p ../jc21tmp pymatgen python==3.8 jupyterlab pip ipython
micromamba activate $(pwd)/.jc21tmp
```

We need to grab a stable `julia` version which we will use `jill.py` for:

```
pip install jill
jill install
```

For all workshops, we need a kernel to work with `jupyterhub` which we get in a straightforward way with an interactive `julia` session.

```
# julia
using Pkg
Pkg.add("IJulia")
```

Exposing Access

To make sure we can access our remote server we can use `ngrok` [^fn:2] or `ssh` forwarding [^fn:3].

```
wget https://bin.equinox.io/c/4VmDzA7iaHb/ngrok-stable-linux-amd64.zip
unzip ngrok-stable-linux-amd64.zip
mv ngrok $HOME/.local/bin
ngrok authtoken <token>
ngrok http 8888
```

We have assumed again, that `$HOME/.local/bin` is in our path. `tmux` is optional, but it is fantastic for all headless setups. It is probably better to not do this on your login / head node, and with `ngrok` we don't need to care about how deeply nested we are in terms of exposing the tunnel, so just grab an interactive job. Assuming `tmux` then, the workflow is simply:

```
ssh mysuperhpc
# 6.5 hours on 1 node
srunk -N 1 -t 6:30:00 --pty /bin/bash
# Or whatever configuration works for you
tmux new -s jc21 # enter an environment
cd $jc21
micromamba activate $(pwd)/.jc21tmp
jupyter lab --ServerApp.allow_remote_access=1 \
  --ServerApp.open_browser=False --port=8889
# Ctrl+B --> : --> split-pane
ngrok http 8889
```

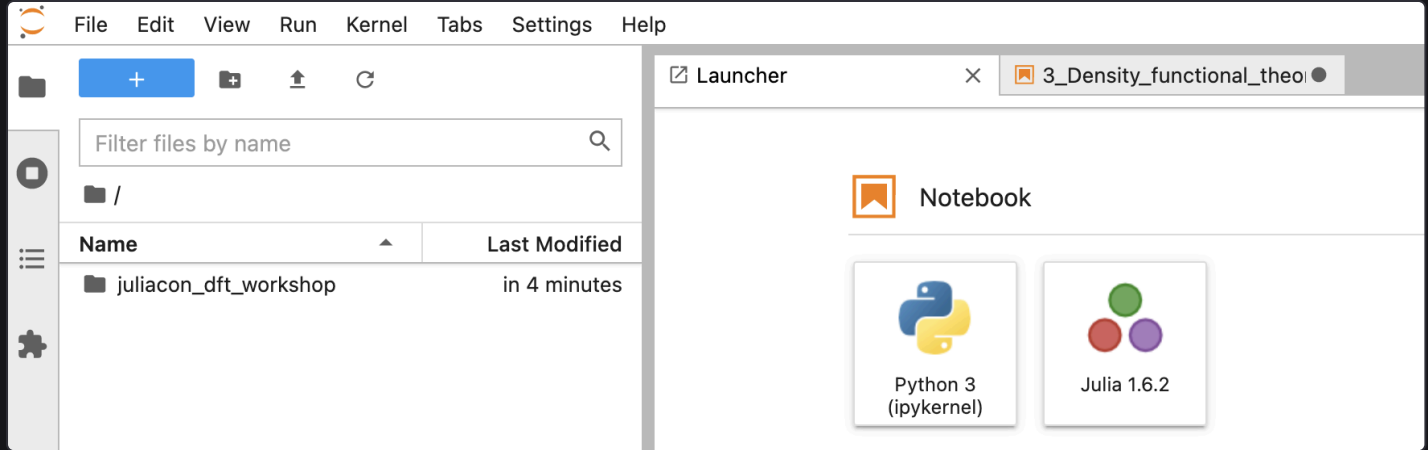
Where the `jupyter lab` options are self evident and circumvent having to generate a configuration file. The second pane does not need to be run from the same location naturally. With this, pointing a local browser anywhere to the `ngrok` URL will bring you control over your remote machine and we're off to the races.

```
or http://127.0.0.1:8889/lab?token=504604e0f6d9f0827475652737fd41394ed94c735399da7c
```

ngrok by @inconshreveable

Session Status	online
Account	Rohit Goswami (Plan: Free)
Update	update available (version 2.3.40, Ctrl-U to update)
Version	2.3.35
Region	United States (us)
Web Interface	http://127.0.0.1:4040
Forwarding	http://0cea293c01b3.ngrok.io -> http://localhost:8889
Forwarding	https://0cea293c01b3.ngrok.io -> http://localhost:8889

Now we get a standard server with the appropriate kernel and can continue working through the workshops.



Conclusions

I'm not jumping on the `julia` bandwagon. I still reach for `C++` or `Fortran` or even `Python`; but JuliaCon is **why** I need to be conversant with `julia`. It is used for fantastic applied use-cases, and being practically able to follow along and hack around existing code is important, even if I'm not interested in it as a programming language per-se. In general these instructions could be generalized to other workflows/workshops/kernels etc. etc., so it helps to have it written down too. Keep in mind that some system admins will take umbrage at running servers like this.